UNIVERSIDAD ICESI

Facultad de Ingeniería, Diseño y Ciencias Aplicadas — Departamento de Computación y Sistemas Inteligentes

Informe Técnico — Taller Guiado

Chat Distribuido por Consola con Middleware ZeroC Ice y Gradle Multimódulo

Estudiante: Mateo Cuenca Mejía

1. Objetivo del taller

El taller busca que dejemos de ver el paso de mensajes por sockets TCP crudos como la única forma de comunicar procesos, y pasemos a un middleware RPC (ZeroC Ice) que resuelve por nosotros la serialización, el tipado y el descubrimiento de objetos remotos a partir de un contrato formal en Slice. La entrega concreta es un chat multiusuario por consola, dividido en tres módulos de Gradle (common, server, client), que demuestre login, envío/recepción de mensajes en tiempo real, consulta de usuarios conectados y manejo de fallos de red.

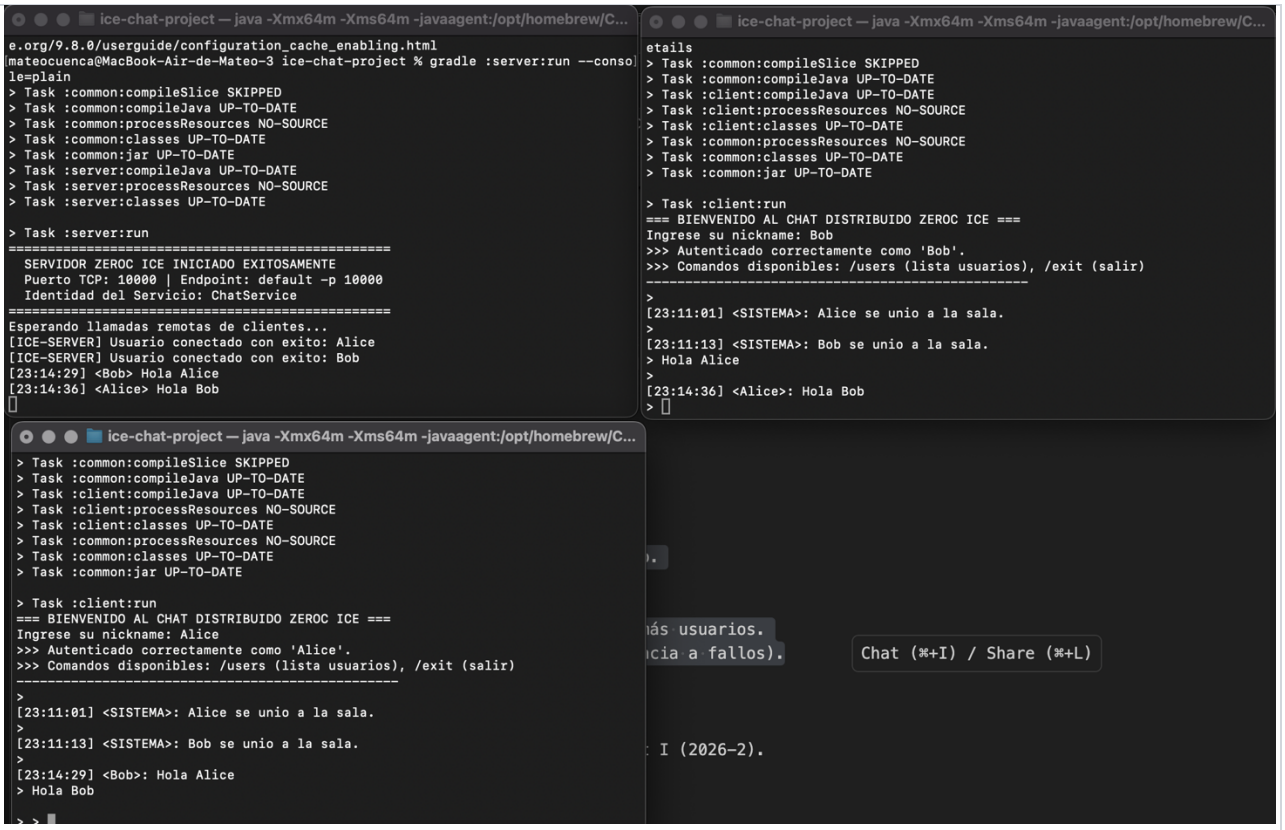
2. Repositorio y ejecución

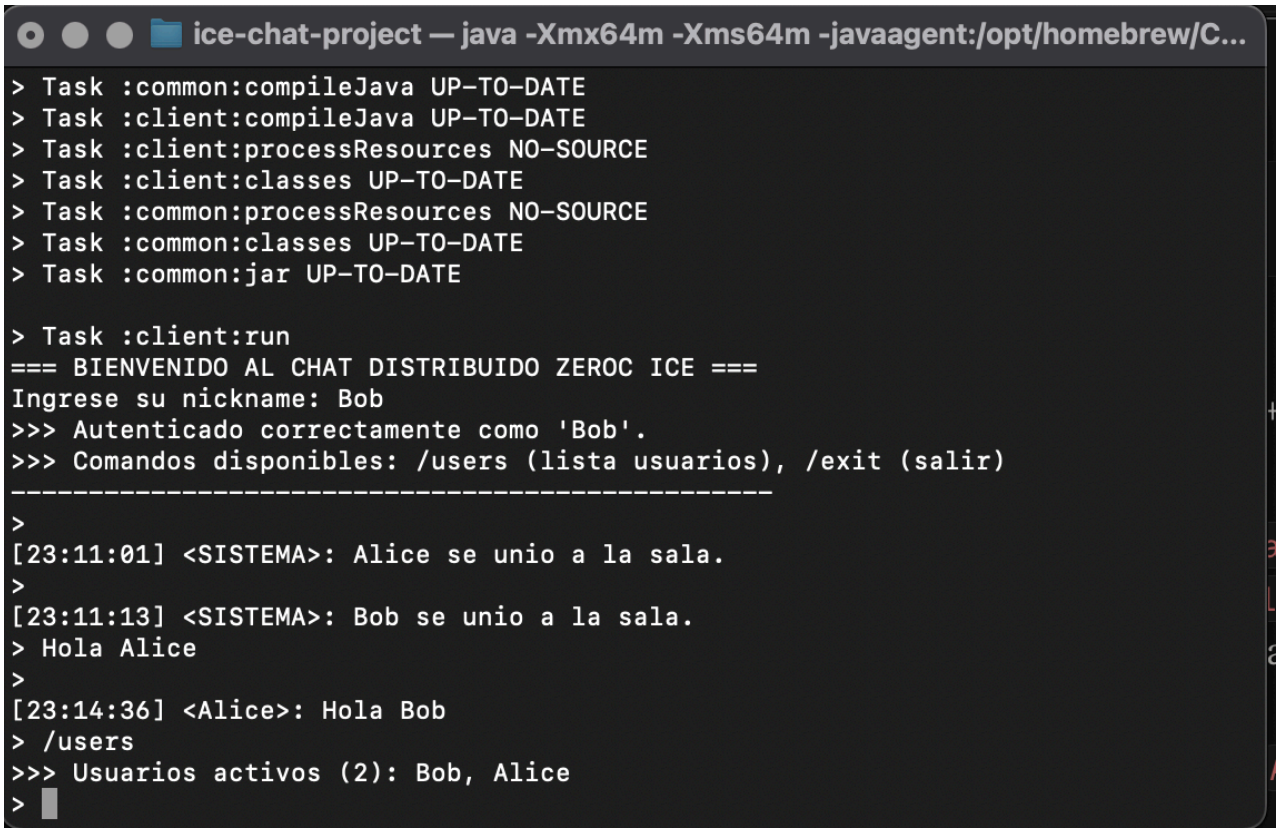
El código fuente completo (contrato Slice, servant, servidor y cliente) está en el repositorio de GitHub del taller, con las instrucciones de compilación y ejecución en el README.md. La estructura sigue exactamente la propuesta en la guía: common/ para el contrato compartido, server/ para el Servant e inicialización del Communicator, y client/ para el REPL interactivo con el hilo demonio de recepción.

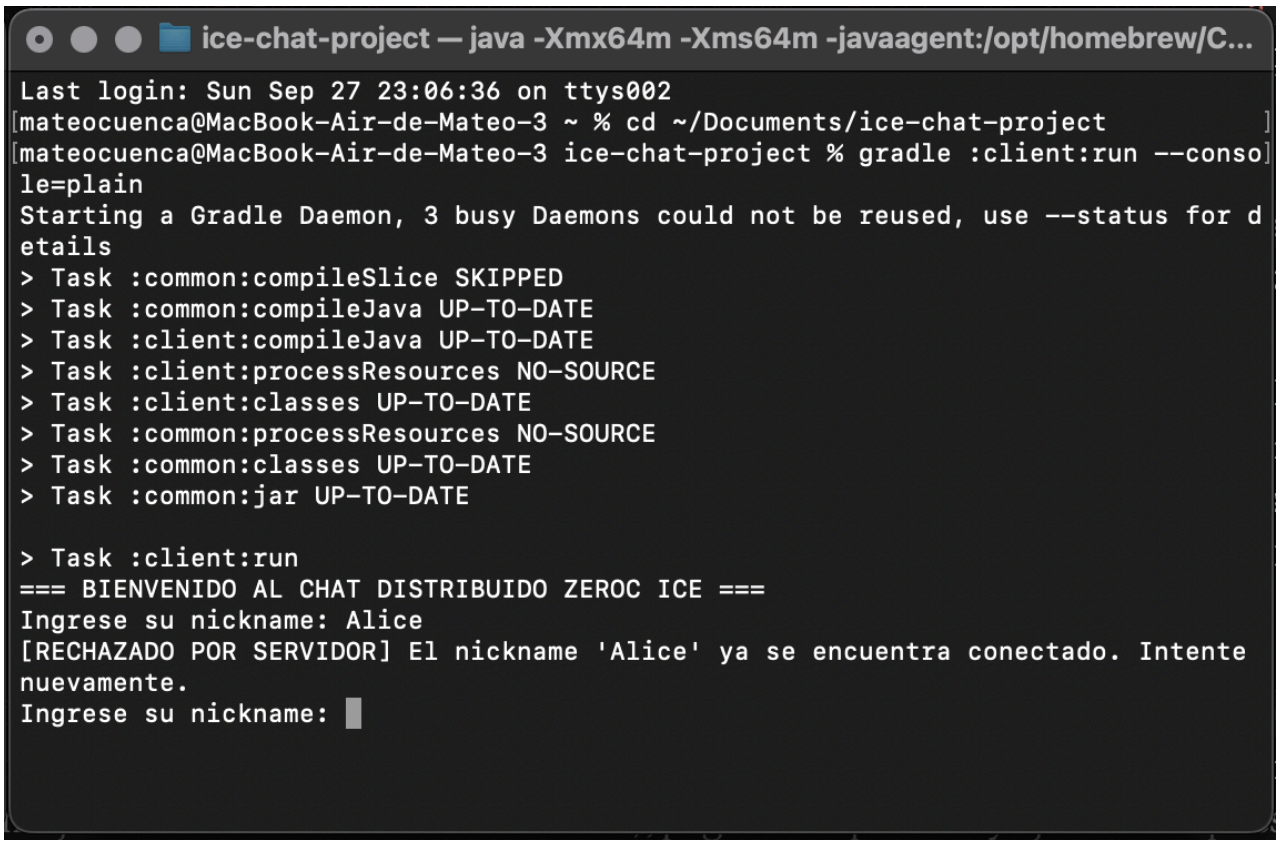
Repositorio: [PEGAR AQUÍ EL ENLACE DEL REPOSITORIO DE GITHUB]

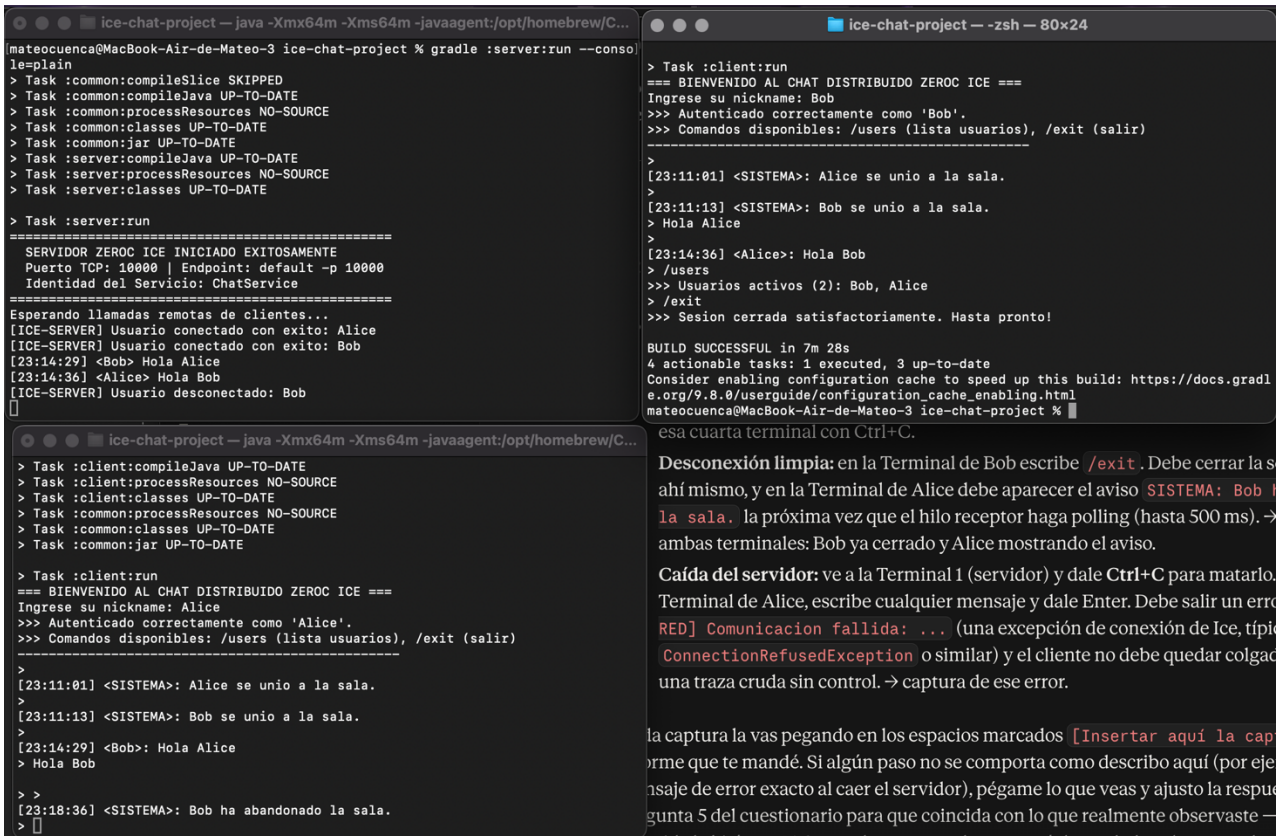
3. Evidencia de ejecución multi-terminal

A continuación se incluyen las capturas de las tres terminales (servidor, Alice y Bob) durante las pruebas obligatorias de la guía: intercambio bidireccional de mensajes, consulta de usuarios con /users, colisión de nickname duplicado, desconexión limpia con /exit y comportamiento del cliente ante la caída del servidor.




ice-chat-project — java -Xmx64m -Xms64m -javaagent:/opt/homebrew/C...
> Task :common:compileJava UP-TO-DATE
> Task :client:compileJava UP-TO-DATE
> Task :client:processResources NO-SOURCE
> Task :client:classes UP-TO-DATE
> Task :common:processResources NO-SOURCE
> Task :common:classes UP-TO-DATE
> Task :common:jar UP-TO-DATE
> Task :client:run
=== BIENVENIDO AL CHAT DISTRIBUIDO ZEROC ICE ===
Ingrese su nickname: Bob
>>> Autenticado correctamente como 'Bob'.
>>> Comandos disponibles: /users (lista usuarios), /exit (salir)
[23:11:01] <SISTEMA>: Alice se unio a la sala.
[23:11:13] <SISTEMA>: Bob se unio a la sala.
> Hola Alice
[23:14:36] <Alice>: Hola Bob
> /users
>>> Usuarios activos (2): Bob, Alice


ice-chat-project — java -Xmx64m -Xms64m -javaagent:/opt/homebrew/C...
Last login: Sun Sep 27 23:06:36 on ttys002
[mateocuenca@MacBook-Air-de-Mateo-3 ~ % cd ~/Documents/ice-chat-project
[mateocuenca@MacBook-Air-de-Mateo-3 ice-chat-project % gradle :client:run --conso]
le=plain
Starting a Gradle Daemon, 3 busy Daemons could not be reused, use --status for d
etails
> Task :common:compileSlice SKIPPED
> Task :common:compileJava UP-TO-DATE
> Task :client:compileJava UP-TO-DATE
> Task :client:processResources NO-SOURCE
> Task :client:classes UP-TO-DATE
> Task :common:processResources NO-SOURCE
> Task :common:classes UP-TO-DATE
> Task :common:jar UP-TO-DATE
> Task :client:run
=== BIENVENIDO AL CHAT DISTRIBUIDO ZEROC ICE ===
Ingrese su nickname: Alice
[RECHAZADO POR SERVIDOR] El nickname 'Alice' ya se encuentra conectado. Intente
nuevamente.
Ingrese su nickname:

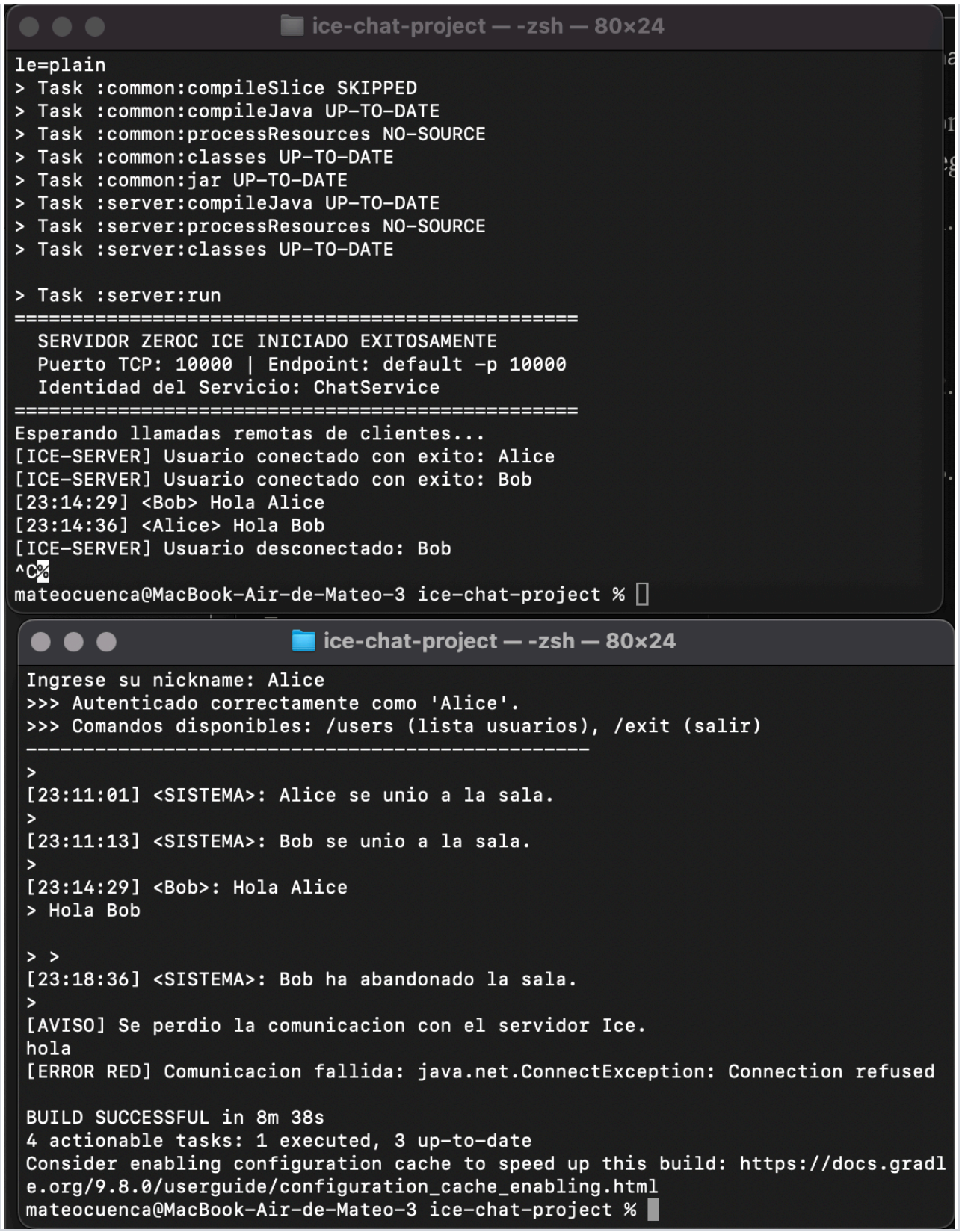


esa cuarta terminal con Ctrl+C.

Desconexión limpia: en la Terminal de Bob escribe `/exit`. Debe cerrar la s
ahí mismo, y en la Terminal de Alice debe aparecer el aviso `SISTEMA: Bob h
la sala.` la próxima vez que el hilo receptor haga polling (hasta 500 ms). →
ambas terminales: Bob ya cerrado y Alice mostrando el aviso.

Caída del servidor: ve a la Terminal 1 (servidor) y dale **Ctrl+C** para matarlo.
Terminal de Alice, escribe cualquier mensaje y dale Enter. Debe salir un erro
`RED] Comunicacion fallida: ...` (una excepción de conexión de Ice, típi
`ConnectionRefusedException` o similar) y el cliente no debe quedar colgad
una traza cruda sin control. → captura de ese error.

la captura la vas pegando en los espacios marcados `[Insertar aquí la capt`
rme que te mandé. Si algún paso no se comporta como describo aquí (por ejem
nsaje de error exacto al caer el servidor), pégame lo que veas y ajusto la respue
gunta 5 del cuestionario para que coincida con lo que realmente observaste —


ice-chat-project — -zsh — 80×24
le=plain
> Task :common:compileSlice SKIPPED
> Task :common:compileJava UP-TO-DATE
> Task :common:processResources NO-SOURCE
> Task :common:classes UP-TO-DATE
> Task :common:jar UP-TO-DATE
> Task :server:compileJava UP-TO-DATE
> Task :server:processResources NO-SOURCE
> Task :server:classes UP-TO-DATE
> Task :server:run
==========
SERVIDOR ZEROC ICE INICIADO EXITOSAMENTE
Puerto TCP: 10000 | Endpoint: default -p 10000
Identidad del Servicio: ChatService
==========
Esperando llamadas remotas de clientes...
[ICE-SERVER] Usuario conectado con exito: Alice
[ICE-SERVER] Usuario conectado con exito: Bob
[23:14:29] <Bob> Hola Alice
[23:14:36] <Alice> Hola Bob
[ICE-SERVER] Usuario desconectado: Bob
^C%
mateocuenca@MacBook-Air-de-Mateo-3 ice-chat-project %
ice-chat-project — -zsh — 80×24
Ingrese su nickname: Alice
>>> Autenticado correctamente como 'Alice'.
>>> Comandos disponibles: /users (lista usuarios), /exit (salir)
>
[23:11:01] <SISTEMA>: Alice se unio a la sala.
>
[23:11:13] <SISTEMA>: Bob se unio a la sala.
>
[23:14:29] <Bob>: Hola Alice
> Hola Bob
> >
[23:18:36] <SISTEMA>: Bob ha abandonado la sala.
>
[AVISO] Se perdio la comunicacion con el servidor Ice.
hola
[ERROR RED] Comunicacion fallida: java.net.ConnectException: Connection refused
BUILD SUCCESSFUL in 8m 38s
4 actionable tasks: 1 executed, 3 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.html
mateocuenca@MacBook-Air-de-Mateo-3 ice-chat-project %

4. Cuestionario de análisis conceptual

4.1 Interoperabilidad de plataformas heterogéneas

El archivo Chat.ice no tendría que cambiar en absoluto. Ese es justamente el punto de tener un contrato Slice: es neutral respecto al lenguaje, así que para levantar un cliente en Python o C++ basta con correr el compilador correspondiente (slice2py o slice2cpp) sobre el mismo .ice y generar los stubs de esa plataforma. El servidor Java sigue exactamente igual.

La interoperabilidad no ocurre a nivel de código sino a nivel de bytes en la red. Ice define una codificación binaria propia (Slice encoding) que especifica de forma exacta cómo se representan los tipos —enteros, strings con su longitud, secuencias, structs, excepciones— independientemente de cómo cada lenguaje los represente en memoria. El compilador slice2java, slice2py o slice2cpp solo genera el "traductor" entre esa representación de red y las estructuras nativas de cada lenguaje; el protocolo IceP que circula por el socket es el mismo sin importar quién lo generó. Por eso un cliente Python y un servidor Java pueden invocar operaciones remotas entre sí sin que ninguno de los dos sepa en qué lenguaje está escrito el otro extremo.

4.2 Rendimiento: IceP vs. REST/JSON

En tamaño de payload, JSON pierde de entrada porque cada mensaje repite los nombres de los campos como texto ("sender":, "text":, etc.) además de comillas y llaves. IceP, al tener el contrato compilado de antemano en ambos extremos, no necesita mandar esos nombres: solo envía los valores en el orden que ya conocen cliente y servidor, lo que en structs como ChatMessage puede representar varias veces menos bytes por mensaje que su equivalente en JSON.

En costo de serialización, un parser JSON tiene que tokenizar texto, construir una estructura intermedia (mapa/diccionario) y después convertirla a objetos tipados, generalmente con reflexión. Ice, en cambio, usa código generado específico para cada tipo que escribe y lee directamente sobre un buffer de bytes, sin pasar por texto ni por reflexión. Para una operación que se llama cada 500 ms como getPendingMessages, esa diferencia de costo por cada llamada sí se nota si se multiplica por muchos clientes.

En multiplexación, HTTP/1.1 depende de mantener varias conexiones TCP abiertas en paralelo para no bloquearse (la mayoría de clientes limitan ese número), porque una conexión sirve una petición a la vez incluso con keep-alive. Ice, al ser un protocolo diseñado para RPC, puede llevar varias invocaciones concurrentes sobre la misma conexión TCP usando un identificador de petición para correlacionar cada respuesta con su llamada correspondiente, sin necesidad de abrir una conexión nueva por cada operación.

4.3 Semántica de idempotencia

getPendingMessages es una consulta que no modifica el estado del servidor: pedirla una vez o pedirla cinco veces devuelve el mismo resultado (los mensajes con id mayor al indicado). Por eso se puede marcar idempotent: si hay un timeout o se pierde un paquete y el runtime de Ice no sabe si la petición llegó a ejecutarse, es seguro reintentarla automáticamente sin riesgo de efectos secundarios.

postMessage sí tiene efecto secundario (agrega un mensaje al historial), así que si el runtime la reintentara automáticamente tras una falla ambigua, se podría terminar publicando el mismo mensaje dos veces. Al no estar marcada idempotent, Ice no la reintenta por su cuenta ante ese tipo

de fallo de red; es responsabilidad del cliente decidir si reenvía o no, evitando así duplicar mensajes solo por una pérdida de paquete.

4.4 ThreadPool vs. hilo por conexión

Con un hilo por conexión, cada cliente nuevo obtiene un hilo del sistema operativo dedicado, que se queda vivo mientras la conexión exista. Si llegan muchas conexiones —ya sea por una carga real o por un ataque deliberado— el número de hilos crece sin límite, cada uno reservando su propia pila de memoria, hasta agotar la memoria disponible o el límite de hilos del sistema operativo, degradando o tumbando el servidor completo (thread starvation).

El ObjectAdapter de Ice, en cambio, despacha las peticiones entrantes hacia un pool de hilos de tamaño acotado y configurable (Ice.ThreadPool.Server.SizeMax), mientras que la aceptación de conexiones y la lectura de sockets se maneja con E/S no bloqueante. El número de hilos del sistema operativo tiene un techo fijo sin importar cuántos clientes se conecten; si llegan más peticiones de las que el pool puede atender en un momento dado, simplemente se encolan en vez de generar hilos nuevos. Eso hace que el consumo de recursos del servidor sea predecible y que un pico de conexiones no pueda agotar la memoria ni los hilos del proceso.

5. Bitácora de uso ético de Inteligencia Artificial Generativa (Nivel 3 — Colaboración Asistida)

Herramienta utilizada: Claude (Anthropic), en su modo de asistente de código (Claude en modo Cowork).

Prompts principales formulados:

Pedí ayuda para: (1) estructurar el proyecto Gradle multimódulo (common/server/client) a partir de la guía del taller y dejar los 9 archivos en las rutas exactas indicadas; (2) revisar el hilo demonio del cliente que hace polling de mensajes cada 500 ms, para confirmar que no bloqueara el hilo principal de lectura de consola; (3) redactar el README con los pasos de compilación e instalación de ZeroC Ice vía Homebrew en macOS; (4) organizar las respuestas del cuestionario conceptual (interoperabilidad, IceP vs REST, idempotencia y thread pooling) para presentarlas de forma clara.

Errores o puntos a corregir detectados durante el proceso:

Al correr gradle build la primera vez, la tarea compileSlice fallaba porque Gradle no encontraba el comando slice2java, a pesar de que brew install ice decía estar instalado y actualizado. Al revisar con which slice2java confirmé que el binario simplemente no existía en el sistema: la fórmula de ice que trae Homebrew (homebrew-core) solo incluye los compiladores para C++, Python y PHP (slice2cpp, slice2py, slice2php), pero no el de Java. Es una limitación de esa fórmula específica, no un error de configuración del proyecto ni de instalación de mi parte.

Como no vale la pena compilar todo Ice desde el código fuente en C++ solo para obtener ese compilador, la solución fue generar el código Java del contrato Chat.ice una sola vez (en otra máquina con slice2java disponible) y dejarlo ya incluido dentro de common/src/main/java/ChatApp/, ajustando common/build.gradle para que la regeneración automática solo se intente si slice2java está presente; si no lo está, usa directamente el código ya generado. Con ese cambio gradle build compiló sin problema.

Estrategia de validación técnica aplicada:

El código generado no se dio por válido solo porque compilara: se ejecutó el flujo completo en tres terminales reales (servidor, Alice, Bob) y se verificaron uno por uno los cinco escenarios de prueba exigidos por la guía

(mensajes bidireccionales, /users, nickname duplicado, /exit y caída del servidor), confirmando que el comportamiento observado coincidiera con lo esperado en el contrato Slice antes de dar por completa cada parte.